

Report – Sudoku Java RMI

❖ Indice

1. Analisi del problema
2. Architettura proposta
3. Consistenza e sincronizzazione
4. Gestione degli eventi dinamici
5. Sviluppo
6. Risultati e considerazioni

❖ 1. Analisi del problema

Il problema è lo stesso della Part 2A — Sudoku cooperativo distribuito — ma la soluzione adotta un approccio **centralizzato** basato su **Java RMI** (Distributed Object Computing).

Grazie a RMI, oggetti Java residenti su macchine diverse comunicano come se fossero locali, con la gestione della serializzazione e del trasporto trasparente per il programmatore.

Il sistema deve:

- Permettere la **creazione e il join dinamico** di stanze di gioco.
- Garantire che tutti i giocatori vedano la griglia in modo **consistente** (happened-before).
- Mostrare in tempo reale le **selezioni** delle caselle degli altri giocatori.
- Gestire l'**uscita/crash** di qualsiasi giocatore, incluso chi ha creato la stanza.

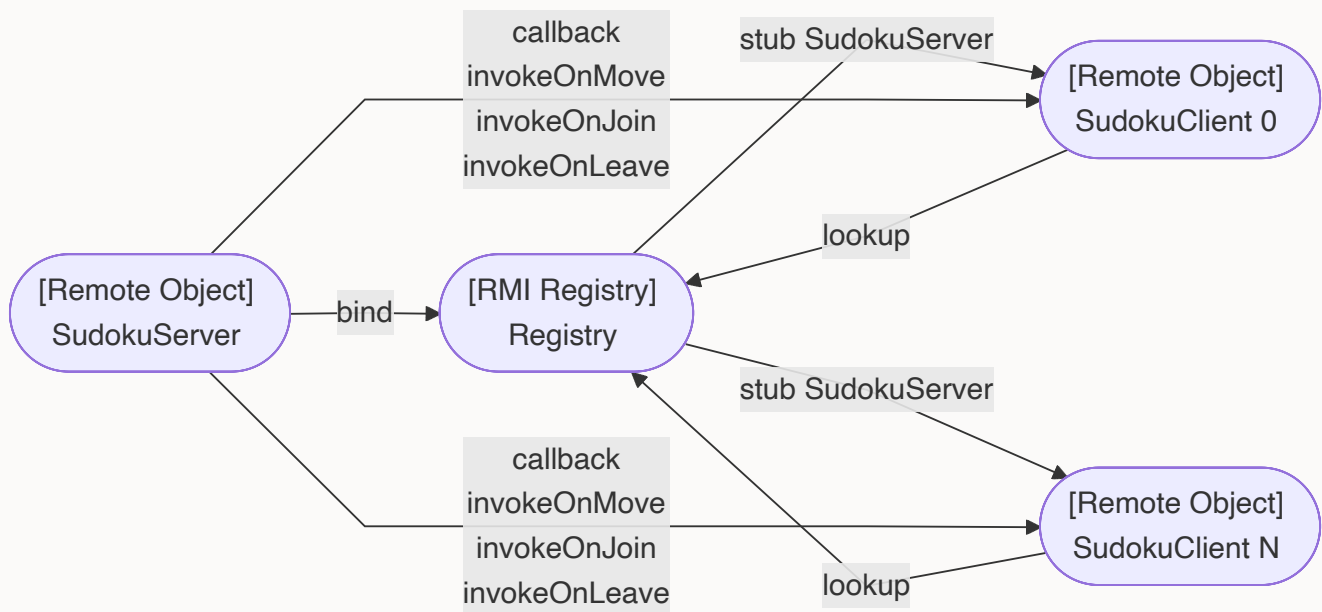
Requisiti chiave

Requisito	Descrizione
R1	Creazione e join dinamico di stanze di gioco
R2	Consistenza happened-before sugli eventi di modifica griglia
R3	Visibilità delle selezioni correnti degli altri giocatori
R4	Uscita/crash di qualsiasi giocatore senza bloccare la partita
R5	Architettura centralizzata (consentita dalla consegna per RMI)

❖ 2. Architettura proposta

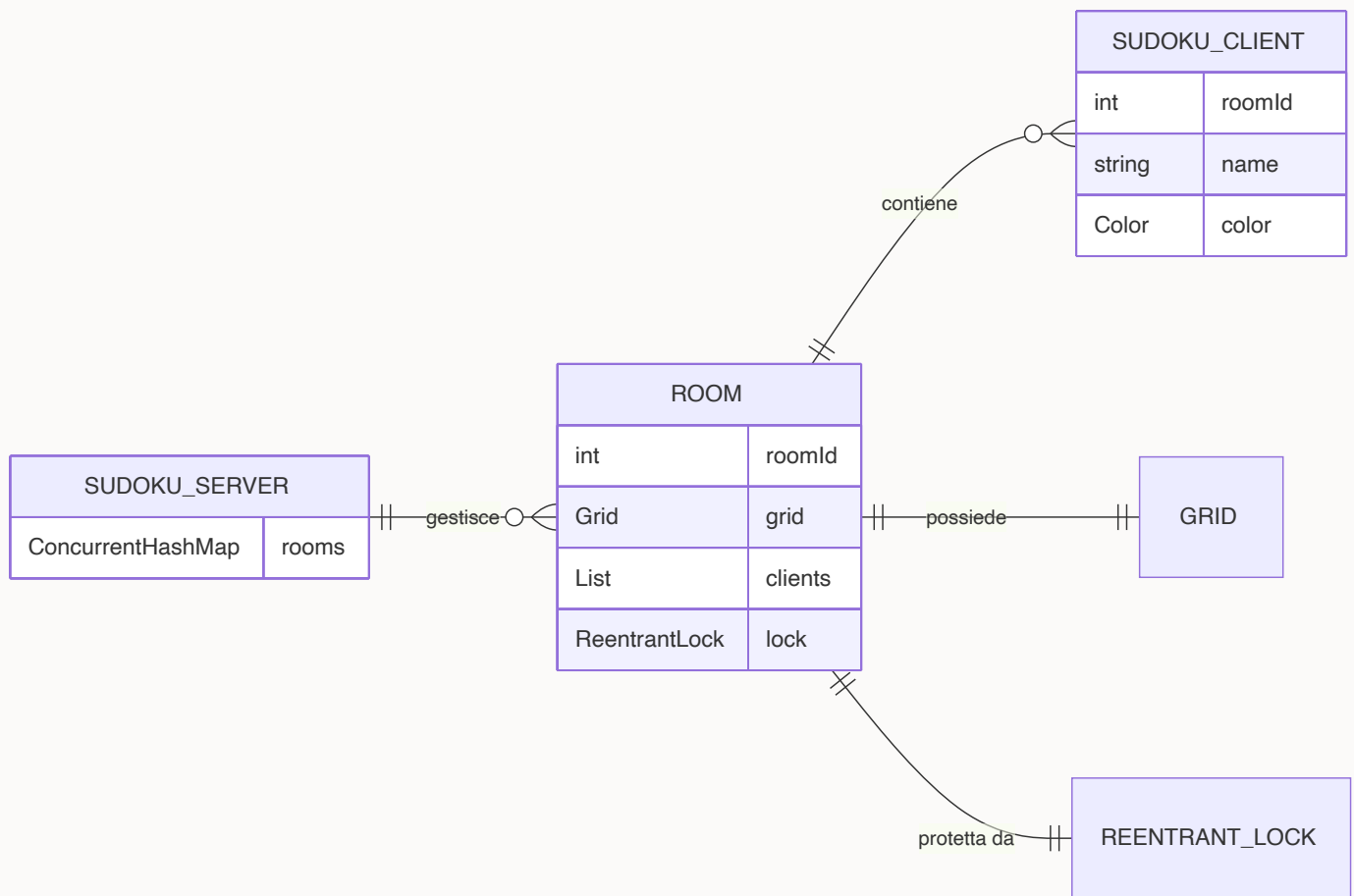
L'architettura è **centralizzata**: un `SudokuServer` remoto gestisce tutte le stanze attive e coordina le interazioni tra i client. Ogni client è a sua volta un oggetto remoto (`SudokuClient`), permettendo al server di notificarli via **callback**.

2.1 Componenti del sistema



- **SudokuServer**: oggetto remoto centralizzato che gestisce le stanze. Mantiene una `ConcurrentHashMap<Integer, Room>` con un `ReentrantLock` per stanza.
- **SudokuClient**: ogni client è un oggetto remoto che espone metodi di callback invocati dal server per notificare aggiornamenti.
- **RMI Registry**: punto di lookup per ottenere lo stub del server.

2.2 Struttura di una stanza



2.3 Tipi di callback server → client

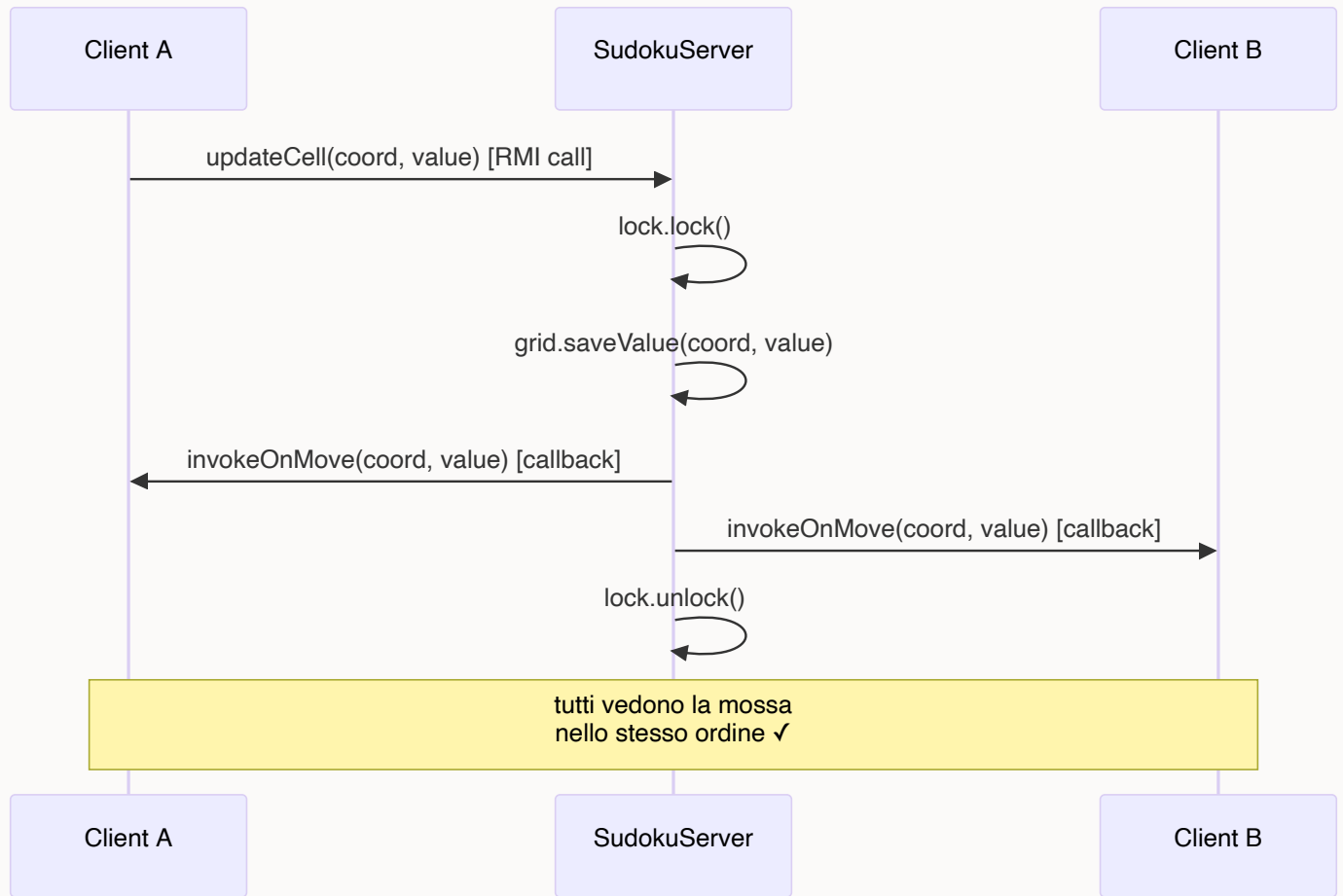
Callback	Scopo
<code>invokeOnEnter</code>	Invia griglia e soluzione al nuovo giocatore
<code>invokeOnMove</code>	Notifica inserimento/modifica di una casella
<code>invokeOnJoin</code>	Invia lista giocatori correnti al nuovo arrivato
<code>invokeOnJoinPlayer</code>	Notifica agli altri l'ingresso di un nuovo giocatore
<code>invokeOnLeavePlayer</code>	Notifica l'uscita di un giocatore
<code>invokeOnFocusGained</code>	Notifica la selezione di una casella
<code>invokeOnFocusLost</code>	Notifica la deselection di una casella

❖ 3. Consistenza e sincronizzazione

3.1 ReentrantLock per stanza (R2)

La consistenza è garantita da un `ReentrantLock` per ogni stanza: tutte le operazioni che modificano la griglia o la lista dei giocatori acquisiscono il lock prima di procedere.

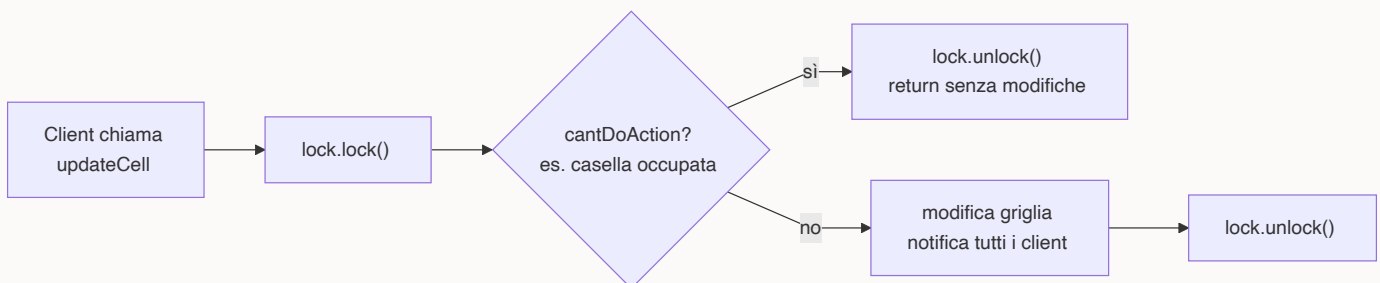
Questo serializza gli aggiornamenti, garantendo happened-before per tutti i client.



Poiché il server notifica tutti i client **dentro il lock**, è impossibile che due client vedano le mosse in ordine diverso: il lock serializza sia la modifica che le notifiche.

3.2 cantDoAction dentro il lock

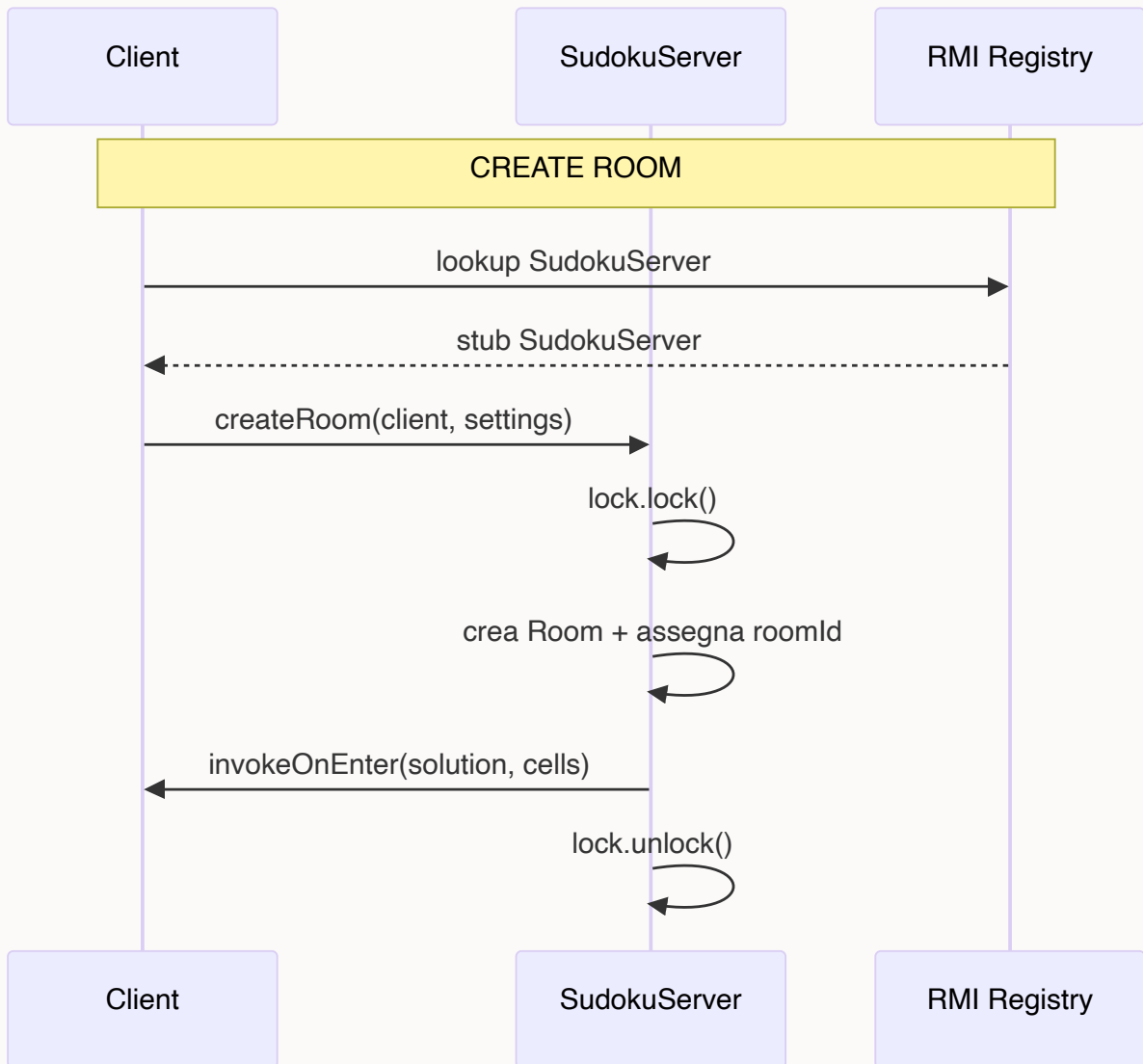
La verifica delle precondizioni (es. casella già occupata, giocatore non autorizzato) avviene **dentro il lock** per evitare race condition di tipo check-and-act:

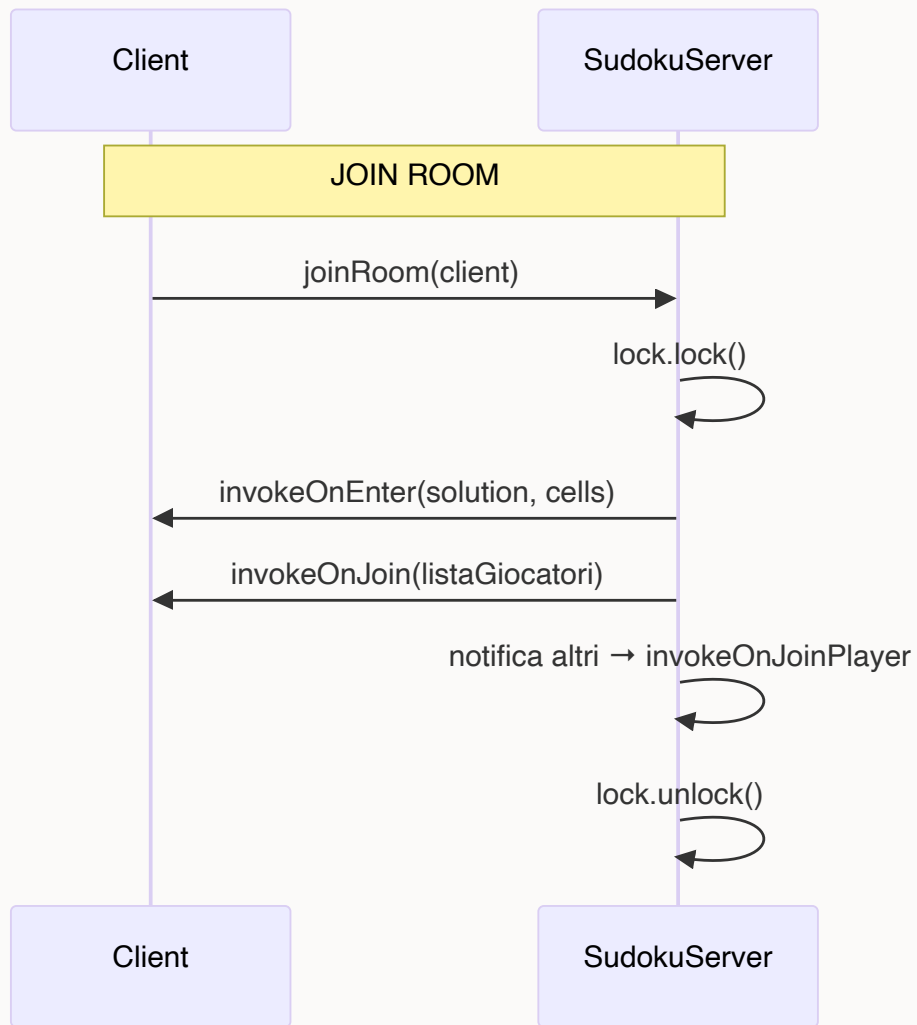


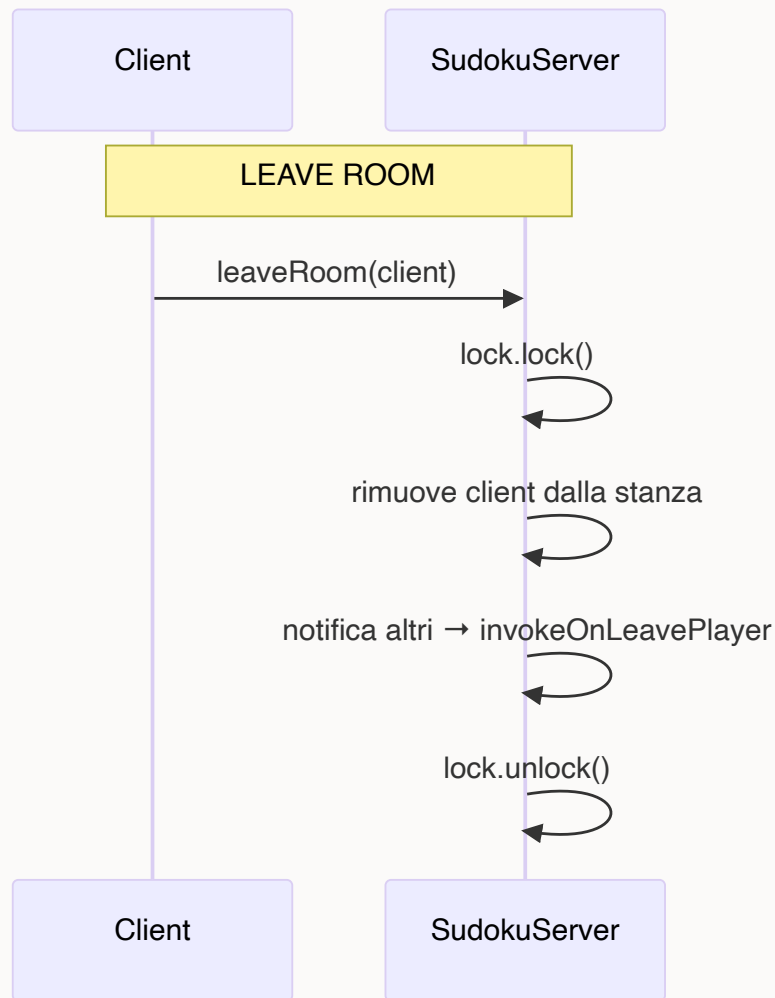
Se `cantDoAction` fosse verificato fuori dal lock, un altro thread potrebbe modificare la griglia tra il check e l'act, causando inconsistenze.

❖ 4. Gestione degli eventi dinamici

4.1 Create, Join e leave volontario (R1, R4)



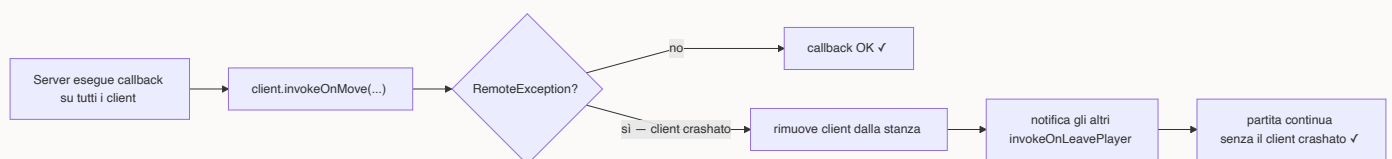




4.2 Crash detection — notifyOrRemove (R4)

Quando il server tenta una callback su un client crashato, RMI lancia una `RemoteException`.

Il metodo `notifyOrRemove` intercetta questa eccezione e rimuove il client dalla stanza, notificando gli altri giocatori:

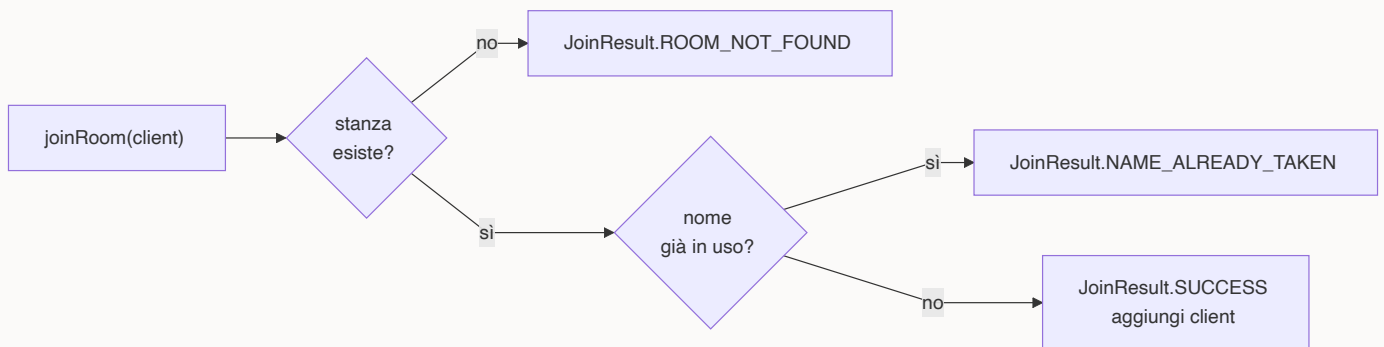


```

1 private void notifyOrRemove(List<SudokuClient> clients, Consumer<SudokuClient> action)
2 {
3     clients.removeIf(client -> {
4         try {
5             action.accept(client);
6             return false; // client OK, non rimuovere
7         } catch (RemoteException e) {
8             return true; // client morto, rimuovilo
9         }
10    });
11 }
  
```

4.3 JoinResult — gestione errori al join

Per gestire i casi di errore al join (stanza non trovata, nome già in uso) è stato definito un enum `JoinResult` che permette messaggi di errore precisi all'utente:



❖ 5. Sviluppo

5.1 SudokuServer

```

1 public interface SudokuServer extends Serializable, Remote {
2     JoinResult createRoom(SudokuClient client, Settings settings) throws RemoteException;
3     JoinResult joinRoom(SudokuClient client) throws RemoteException;
4     void leaveRoom(SudokuClient client) throws RemoteException;
5     void updateCell(SudokuClient client, Coordinate coordinate, int value) throws
6     RemoteException;
7     void focusGained(SudokuClient client, Coordinate coordinate) throws RemoteException;
8     void focusLost(SudokuClient client, Coordinate coordinate) throws RemoteException;
9     byte[][] solution(SudokuClient client) throws RemoteException;
10    byte[][] grid(SudokuClient client) throws RemoteException;
11 }

```

5.2 SudokuClient

```

1 public interface SudokuClient extends Serializable, Remote {
2     int roomId() throws RemoteException;
3     String name() throws RemoteException;
4     void setRoomId(int roomId) throws RemoteException;
5     void invokeOnEnter(byte[][] solution, byte[][] cells) throws RemoteException;
6     void invokeOnMove(Coordinate coordinate, int value) throws RemoteException;
7     void invokeOnJoin(List<String> players) throws RemoteException;
8     void invokeOnJoinPlayer(String player, Color color) throws RemoteException;
9     void invokeOnLeavePlayer(String player) throws RemoteException;
10    void invokeOnFocusGained(String player, Color color, Coordinate coordinate) throws
11    RemoteException;
12    void invokeOnFocusLost(String player, Coordinate coordinate) throws RemoteException;
13 }

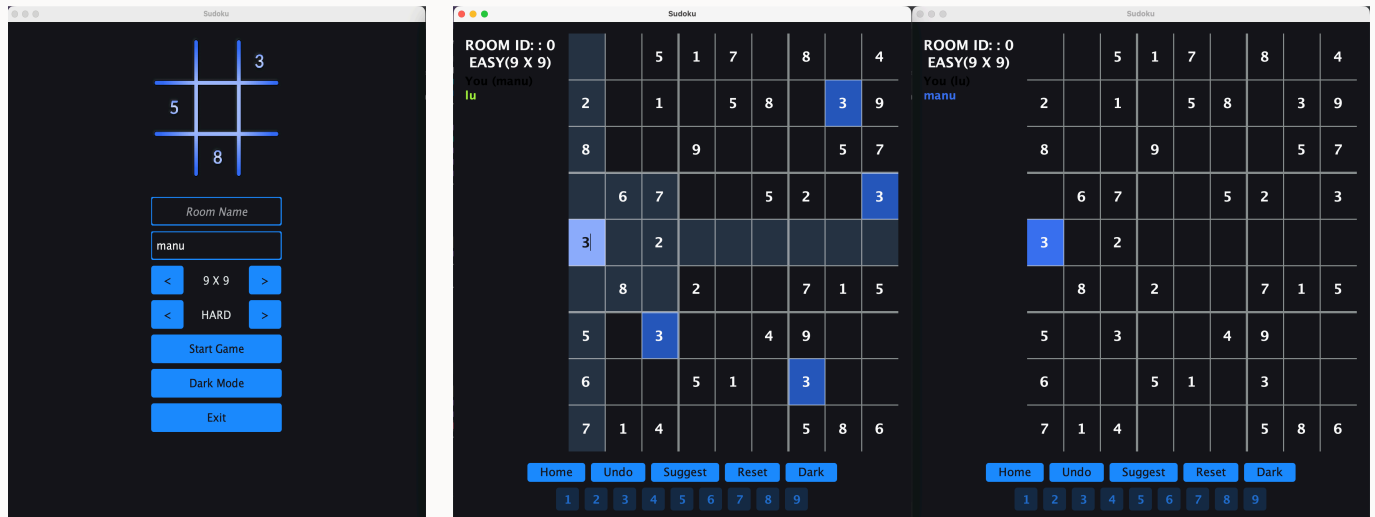
```

5.3 ClientDatas

`ClientDatas` è un record serializzabile che il server usa per identificare un client senza mantenere un riferimento diretto all'oggetto remoto durante la serializzazione:

```
1 public record ClientDatas(int roomId, String name, Color color) implements Serializable
2 {
3 }
```

5.4 Interfaccia grafica



La GUI usa `JTextPane` con `StyledDocument` per mostrare i messaggi di gioco con il colore specifico di ogni giocatore. Il colore viene assegnato dal server al join (`setColor` chiamato prima della serializzazione di `ClientDatas`) e propagato a tutti i client tramite `invokeOnJoinPlayer`.

❖ 6. Risultati e considerazioni

Vantaggi dell'approccio RMI centralizzato

- **Consistenza semplice:** il `ReentrantLock` per stanza serializza tutti gli accessi — nessun algoritmo distribuito necessario. La complessità di sincronizzazione è tutta nel server.
- **Crash detection nativa:** `RemoteException` segnala automaticamente la disconnessione di un client — nessuna infrastruttura aggiuntiva (no heartbeat, no autoDelete).
- **Trasparenza RMI:** le chiamate remote appaiono come chiamate locali, semplificando lo sviluppo rispetto a protocolli di messaggistica espliciti.
- **Callback bidirezionale:** il server può notificare i client attivamente senza polling, grazie al fatto che ogni client è a sua volta un oggetto remoto.

Svantaggi dell'approccio RMI centralizzato

- **Single point of failure:** se il server va giù, tutte le partite attive sono perse. Nell'approccio MOM, il broker è l'unico punto critico ma gestisce la persistenza dei messaggi; qui la griglia esiste solo in memoria sul server.

- **Scalabilità limitata:** il lock per stanza serializza le operazioni — con molti giocatori nella stessa stanza, le callback sequenziali aumentano la latenza percepita.

Confronto con Part 2A (MOM)

Aspetto	RMI (centralizzato)	MOM (decentralizzato)
Consistenza	ReentrantLock sul server	FIFO fanout exchange
Crash detection	RemoteException	autoDelete + cancellation callback
Single point of failure	Server applicativo	Broker RabbitMQ
Complessità	Bassa (lock + callback)	Media (self-filter, gridRequest)
Scalabilità	Limitata dal lock	Migliore (broker gestisce il routing)